Object-Oriented Programming: Complete Deep Dive

This is going to be a comprehensive explanation of every concept from the video. Let's go concept by concept, building from the ground up.

---

1. PILLARS OF OOP — The Big Picture

Before diving in, understand *why* OOP exists. In procedural programming, data and functions are separate. You have variables floating around, and functions that operate on them — no relationship, no structure.

OOP came along and said: **"In the real world, objects have their own data AND their own behavior bundled together. Let's code the same way."**

A car has data (color, speed, fuel level) and behavior (start, brake, accelerate). These don't exist separately — they belong to the car. OOP mimics this.

The four pillars that make OOP powerful are:

- **Encapsulation** — bundling + protecting data
- **Inheritance** — reusing and extending existing code
- **Abstraction** — hiding complexity
- **Polymorphism** — one interface, many behaviors

Today we cover the first two deeply.

---

2. ENCAPSULATION — Complete Deep Dive

What does the word mean?

The word "encapsulation" contains "capsule." Think of a medicine capsule — it has multiple compartments inside, all wrapped in a protective outer shell. You can't directly touch what's inside.

Encapsulation in OOP means exactly this: **wrapping data and behavior inside a protective shell (a class), and controlling how the outside world interacts with that data.**

The Two Rules of Encapsulation

Encapsulation has exactly **two core principles**:

Rule 1: Data and behavior must be bundled together inside an object.

Every real-world object has both data and behavior intrinsically. A dog has data (breed, age, name) and behavior (eat, run, bark). These aren't external — they *belong* to the dog.

So in code, when you represent an object, both its data (variables) and behavior (methods) must live inside the same class. This is something we already do naturally:

```
class Animal {
   // DATA
   int numberOfLegs;
   String type; // carnivore, herbivore

   // BEHAVIOR
   void eat() { ... }
   void walk() { ... }
   void run() { ... }
}
```

Everything belonging to Animal is inside Animal. This is the first half of encapsulation — and most programmers do this without even thinking about it.

Rule 2: You must NOT provide unrestricted access to your data.

This is where most beginners miss the point. Just because data is *inside* a class doesn't mean it's *protected*. Without protection, anyone can reach in and change anything — which breaks real-world logic.

The Bank Account Problem — Understanding Why Rule 2 Exists

Consider this class:

```
class BankAccount {
   double balance;
}
```

Now in your main code:

```
BankAccount ba = new BankAccount();
ba.balance = 1000000; // Set balance to 10 lakh directly!
```

This compiles. This runs. But **this is catastrophically wrong.**

In the real world, can you walk into a bank and just tell them "set my balance to 10 lakh"? Absolutely not. Your balance only changes through specific, validated operations:

- **Deposit** — you physically bring money, they verify it, then add it
- **Withdrawal** — you request an amount, they check if you *have* that amount, then subtract it

The bank never lets you directly modify the balance field. There's always a controlled operation in between.

So why should your code be any different?

The problem with direct access:

- No validation (can set balance to -500000)
- No logging or tracking of changes
- No business rules enforced
- Anyone can change anything anytime — complete chaos

The Solution: Access Modifiers + Methods

This leads us to **access modifiers** — keywords that control who can access what.

---

3. ACCESS MODIFIERS — Complete Deep Dive

Access modifiers answer one question: **"Who is allowed to access this variable, method, constructor, or class?"**

Java has **four** access modifiers. Think of them on a spectrum from most restrictive to least restrictive:

```
PRIVATE → DEFAULT → PROTECTED → PUBLIC
(most locked)                   (fully open)
```

Let's understand each one deeply.

ACCESS MODIFIER 1: `private`

Definition: Only accessible within the same class. Completely invisible to the outside world.

```
class BankAccount {
    private double balance; // No one outside can touch this

    void someMethod() {
        balance = 500; // This works — we're INSIDE the class
    }
}

class Main {
    public static void main(String[] args) {
        BankAccount ba = new BankAccount();
        ba.balance = 1000; // COMPILE ERROR — balance is private!
    }
}
```

The moment you write `private`, it's like putting data in a vault. Only code *inside that same class* can open the vault.

Why is this powerful?

Because you control *how* data is changed. Instead of direct access, you force the outside world to go through methods you've defined — methods where you can add validation, logging, business rules, etc.

ACCESS MODIFIER 2: `default` (no keyword)

Definition: Accessible by any class in the **same package**. When you write no access modifier at all, Java uses `default` automatically.

```
class Animal {
    int numberOfLegs; // No modifier = default
    String name;      // Also default
}
```

What is a package?

A package is simply a **folder** that groups related classes together. Think of it like organizing files on your computer — you put all "school" related files in a "school" folder.

```
📁 school/
   Student.java    → package school
   Teacher.java    → package school

📁 college/
   Student.java    → package college
   Professor.java  → package college
```

In Java, you declare a package at the top of your file:

```
package school; // This file belongs to the "school" package

class Student {
   // code
}
```

Default access means: Any class inside the `school` package can access this field. But a class in the `college` package cannot.

```
// Inside school package
class Teacher {
   void someMethod() {
      Student s = new Student();
      s.name = "Rahul"; // Works — same package
   }
}

// Inside college package
class Professor {
   void someMethod() {
      school.Student s = new school.Student();
      s.name = "Rahul"; // ERROR — different package, default not accessible
   }
}
```

ACCESS MODIFIER 3: `protected`

Definition: Accessible within the **same package** OR in any **subclass** (child class), even if the subclass is in a different package.

This modifier makes full sense only when you understand inheritance, but here's the idea:

```
class Animal {
    protected String name; // Same package classes + child classes can access
}

// In a different package, but inherits from Animal
class Dog extends Animal {
    void display() {
        System.out.println(name); // Works! Dog is a child of Animal
    }
}
```

`protected` is specifically designed for **inheritance scenarios** where a child class needs access to parent class members, even across packages.

ACCESS MODIFIER 4: `public`

Definition: Accessible by absolutely everyone, from anywhere in the entire codebase.

```
class BankAccount {
    public String bankName; // Anyone, anywhere can access this
}
```

No restrictions whatsoever. This is the most open modifier.

Visual Summary of Access Modifiers

Modifier	Same Class	Same Package	Subclass (other package)	Everyone
`private`	✅	❌	❌	❌
`default`	✅	✅	❌	❌

`protected`	✅	✅	✅	❌
`public`	✅	✅	✅	✅

Real-world usage: In production code, you'll almost exclusively use `private` and `public`. `default` and `protected` are rare. `private` for data, `public` for methods that need to be called from outside.

Access Modifiers on Classes

You can put access modifiers on classes too, but with restrictions.

For **top-level classes** (root-level, not nested inside another class), you can only use:

- `public` — accessible from anywhere
- `default` — accessible only within the package

You **cannot** use `private` or `protected` on a top-level class. Why?

Why not `private` on a class? `private` means "only accessible within the enclosing class." But a top-level class has nothing enclosing it — it *is* the outermost thing. There's nothing for it to be private *to*. Saying "private class Animal" is like asking "private compared to what?" It makes no logical sense.

Why not `protected` on a class? `protected` applies to inheritance — it allows child classes to access members. But inheritance applies to *members inside a class* (variables, methods), not to an entire class itself. You can't really "inherit" a whole class the same way.

So for top-level classes, stick to `public` or nothing (default).

(Note: nested/inner classes — a class defined inside another class — can be `private`. We'll cover that later.)

---

4. IMPLEMENTING ENCAPSULATION — Complete Deep Dive

Now let's put it all together and actually implement encapsulation properly.

Step 1: Make all fields `private`

```
class BankAccount {
    private double balance;
    private String bankName;
    private String ifscCode;
}
```

Now no one can directly read or modify these. The vault is locked.

Step 2: Provide controlled access through `public` methods

```
class BankAccount {
    private double balance;

    // Controlled way to increase balance
    public void deposit(double amount) {
        if (amount <= 0) {
            System.out.println("Invalid amount");
            return;
        }
        balance += amount;
        System.out.println("Deposited: " + amount);
    }

    // Controlled way to decrease balance
    public void withdraw(double amount) {
        if (amount > balance) {
            System.out.println("Insufficient funds");
            return;
        }
        if (amount <= 0) {
            System.out.println("Invalid amount");
            return;
        }
        balance -= amount;
        System.out.println("Withdrawn: " + amount);
    }

    // Read-only access to balance
    public double getBalance() {
        return balance;
    }
}
```

Now in main:

```
BankAccount ba = new BankAccount();
ba.balance = 1000000;        // COMPILE ERROR — private!
ba.deposit(500);             // Works — controlled!
ba.withdraw(300);            // Works — with validation!
System.out.println(ba.getBalance()); // 200.0
```

Why is this better than just making balance public?

Because now every single operation on balance goes through YOUR code, where you can:

- Validate input (no negative deposits)
- Check business rules (can't withdraw more than balance)
- Log every transaction
- Update connected systems (send SMS notification)
- Prevent illegal states (balance can never go below 0)

With a public field, balance could be set to -9999999 and Java wouldn't care. With encapsulation, that's impossible.

Getters and Setters

Getters and setters are the most common pattern in encapsulation.

Getter — a method that *reads* a private field and returns it:

```
private String name;

public String getName() {   // getter
    return name;
}
```

Setter — a method that *writes* to a private field, with validation:

```
private int age;

public void setAge(int age) {   // setter
    if (age < 0 || age > 150) {
        System.out.println("Invalid age");
```

```
        return;
    }
    this.age = age;
}
```

Full example with a Student class:

```
class Student {
    private String name;
    private int rollNumber;
    private int age;
    private String college;

    // Constructor
    public Student(String name, int rollNumber, int age, String college) {
        this.name = name;
        this.rollNumber = rollNumber;
        this.age = age;
        this.college = college;
    }

    // Getters
    public String getName() { return name; }
    public int getRollNumber() { return rollNumber; }
    public int getAge() { return age; }
    public String getCollege() { return college; }

    // Setters with validation
    public void setName(String name) {
        // Validation: name must have both first and last name
        if (!name.contains(" ")) {
            System.out.println("Please provide full name (first + last)");
            return;
        }
        this.name = name;
    }

    public void setAge(int age) {
        // Validation: age must be reasonable
        if (age < 5 || age > 100) {
            System.out.println("Invalid age");
            return;
        }
        this.age = age;
```

```
    }

    public void setCollege(String college) {
        // Validation: could verify against a list of real colleges
        // For now, just ensuring it's not empty
        if (college == null || college.isEmpty()) {
            System.out.println("College name cannot be empty");
            return;
        }
        this.college = college;
    }

    public void setRollNumber(int rollNumber) {
        // Validation: roll numbers are 1-100
        if (rollNumber < 1 || rollNumber > 100) {
            System.out.println("Invalid roll number");
            return;
        }
        this.rollNumber = rollNumber;
    }
}
```

Why not just make the field public?

"If I have both a getter and setter, isn't that the same as making it public?"

No — and this is crucial to understand.

With a public field:

- `student.age = -50` — compiles, runs, creates invalid state
- No validation possible
- No way to add logging later without changing all code that uses it

With getter/setter:

- `student.setAge(-50)` — your setter catches it, prints error, rejects it
- You can add logging inside the setter any time, without changing external code
- You can make something read-only by providing a getter but NO setter
- You have complete control over what values are acceptable

The key insight: **sometimes you provide a getter but no setter** (like `getBalance()` in BankAccount). This makes the field read-only from the outside — you can see the balance, but you can't set it directly. You must go through `deposit` or `withdraw`.

---

5. JAVA PACKAGES — Complete Deep Dive

What is a Package?

A package in Java is a **namespace** — a way to organize related classes and avoid naming conflicts. Physically, it's a folder. Conceptually, it's a module or namespace.

```
📁 com/
   📁 myapp/
      📁 school/
         Student.java     → package com.myapp.school
         Teacher.java     → package com.myapp.school
      📁 college/
         Student.java     → package com.myapp.college
         Professor.java   → package com.myapp.college
```

Both packages have a `Student` class — no conflict because they're in different namespaces.

Declaring a Package

The very first line of your Java file must be the package declaration:

```
package school;   // Must be first line

public class Student {
   // code
}
```

If you're in an IDE and you create a file inside a folder, the IDE automatically adds the package declaration for you.

Why Packages?

1. **Organization** — Group related classes (all database-related classes in one package, all UI-related in another)
2. **Naming conflict resolution** — Two different `Student` classes can coexist in different packages
3. **Access control** — `default` modifier works at package level
4. **Reusability** — Share entire packages as libraries

Importing Classes from Other Packages

To use a class from a different package, you must `import` it:

```
import college.Student;   // Import specific class

public class Main {
    public static void main(String[] args) {
        Student s = new Student();   // Now Java knows which Student
        s.print();
    }
}
```

Import with wildcard (*):

```
import college.*;   // Import ALL classes from the college package
```

This imports `Student`, `Teacher`, `Professor` — everything in the `college` package.

Using full qualified name (no import needed):

```
college.Student s1 = new college.Student();
school.Student s2 = new school.Student();
```

You can use the full package path directly without an import statement.

What happens when two imported classes have the same name?

```
import college.Student;
import school.Student;   // CONFLICT! Java doesn't know which Student
```

This causes a compile error. You must use the full qualified name:

```
import college.Student;   // Only import one

college.Student s1 = new college.Student();
school.Student s2 = new school.Student();   // Use full name for the other
```

Built-in Java Packages

Java comes with hundreds of pre-built packages:

Package	Contains
`java.lang`	String, Math, Object, System — auto-imported
`java.util`	ArrayList, HashMap, Scanner, Collections
`java.io`	File, InputStream, OutputStream
`java.net`	Socket, URL, networking classes
`java.sql`	Database connectivity classes

```
import java.util.ArrayList;   // Import ArrayList from java.util
import java.util.Scanner;     // Import Scanner for user input

// Or import everything from util
import java.util.*;
```

`java.lang` is special — it's **automatically imported** in every Java file. That's why you can use `String`, `System`, `Math` without any import statement.

How JVM Handles Imports Internally

When you write `import java.util.ArrayList`:

1. Java compiler looks for `ArrayList.class` (the compiled bytecode) in the `java.util` package
2. It loads that bytecode into memory
3. Now when your code uses `ArrayList`, the JVM knows exactly which class to use
4. When you call methods on ArrayList objects, the JVM can execute the right bytecode

This is why imports must be at the top — the compiler needs to resolve all class names before compiling your code.

Package Naming Convention in Production

In real-world projects, packages follow a **reverse domain name** convention:

com.companyname.projectname.modulename

Example: Google's packages start with `com.google`, Facebook's with `com.facebook`, etc.

```
package com.amazon.payments.gateway;

public class PaymentProcessor {
    // ...
}
```

This ensures global uniqueness — no two companies (with different domain names) can have the same package name.

---

6. INHERITANCE — Complete Deep Dive

What is Inheritance?

Inheritance models the **"is-a" relationship** between objects.

- A Car **is-a** Vehicle ✅
- A Dog **is-a** Animal ✅
- An EngineeringStudent **is-a** Student ✅
- A Manager **is-a** Employee ✅

But NOT:

- A Vehicle **is-a** Car ❌ (not always true)
- A Car **is-a** Dog ❌ (nonsense)

Why Does This Matter in Code?

Without inheritance, if you had 10 types of vehicles (Car, Truck, Motorcycle, Bus, etc.), you'd write the `start()` method 10 times. If you need to change how starting works, you change it in 10 places.

With inheritance, you write `start()` once in `Vehicle`. All 10 subclasses automatically have it. Change it once, it's updated everywhere.

This is code reusability — one of the most powerful benefits of OOP.

Terminology

Term	Meaning

Parent class / **Super class**	The class being inherited from
Child class / **Sub class**	The class that inherits
`extends`	Java keyword to establish inheritance

The `extends` Keyword

```
class Vehicle {
    void start() {
        System.out.println("Vehicle starting...");
    }

    void stop() {
        System.out.println("Vehicle stopping...");
    }
}

class Car extends Vehicle {
    // Car automatically has start() and stop() from Vehicle

    void setGear(int gear) {
        System.out.println("Setting gear to: " + gear);
    }
}

class Motorcycle extends Vehicle {
    // Motorcycle also automatically has start() and stop()

    void doWheelie() {
        System.out.println("Doing a wheelie!");
    }
}

Car c = new Car();
c.start();       // Works! Inherited from Vehicle
c.setGear(3);    // Works! Defined in Car

Motorcycle m = new Motorcycle();
m.start();       // Works! Inherited from Vehicle
m.doWheelie();   // Works! Defined in Motorcycle
```

```
// But:
c.doWheelie();   // ERROR! Car doesn't have doWheelie
m.setGear(3);    // ERROR! Motorcycle doesn't have setGear
```

Each subclass gets everything from the parent, plus its own specific things.

What Gets Inherited?

A child class inherits:

- All `public` methods from parent
- All `protected` methods from parent
- All `public` variables from parent
- All `protected` variables from parent

A child class does NOT inherit:

- `private` methods/variables (they're locked inside the parent only)
- Constructors (constructors are NOT inherited, though they can be called with `super`)

Implementing Inheritance — Student Example

```
class Student {
   public String name;
   public int age;
   public int rollNumber;

   public void markAttendance() {
      System.out.println("Attendance marked for: " + name);
   }
}

class EngineeringStudent extends Student {
   // Automatically has: name, age, rollNumber, markAttendance()

   public String college;

   public void attendLab() {
      System.out.println("Lab attended at: " + college);
   }
}

class MedicalStudent extends Student {
   // Automatically has: name, age, rollNumber, markAttendance()
```

```
    public String hospital;

    public void attendLab() {
        System.out.println("Medical lab attended at: " + hospital);
    }
}

EngineeringStudent es = new EngineeringStudent();
es.name = "Aditya";          // Inherited from Student
es.rollNumber = 101;          // Inherited from Student
es.college = "IIT Bombay";    // Own field
es.markAttendance();          // Inherited method
es.attendLab();               // Own method

Student s = new Student();
s.markAttendance();    // Works
s.attendLab();         // ERROR! Student doesn't have attendLab
```

Access Modifiers and Inheritance

```
class Student {
    public String name;       // Accessible everywhere
    protected int rollNumber; // Accessible in same package + subclasses
    String grade;             // Default — same package only
    private double salary;    // Private — only inside Student class
}

class EngineeringStudent extends Student {
    void display() {
        System.out.println(name);       // Works — public
        System.out.println(rollNumber); // Works — protected
        System.out.println(grade);      // Works — same package
        System.out.println(salary);     // ERROR — private!
    }
}
```

`private` members are like the parent's personal diary — even children can't read it directly.

---

7. TYPES OF INHERITANCE — Complete Deep Dive

Type 1: Single Inheritance

One parent, one child. Simple and clean.

```
A
 |
 B
```

```
class Animal { }
class Dog extends Animal { }
```

This is the most basic form. `Dog` gets everything from `Animal`.

Type 2: Multilevel Inheritance

A chain: A → B → C. B inherits from A, C inherits from B.

```
A
 |
 B
 |
 C
```

```
class Student {
    public void markAttendance() {
        System.out.println("Attendance marked");
    }
}

class EngineeringStudent extends Student {
    public void attendLab() {
        System.out.println("Lab attended");
    }
}

class CSEEngineeringStudent extends EngineeringStudent {
    public void attendCSELab() {
        System.out.println("CSE lab attended");
    }
}

CSEEngineeringStudent cse = new CSEEngineeringStudent();
```

```
cse.markAttendance();  // From Student (grandparent)
cse.attendLab();       // From EngineeringStudent (parent)
cse.attendCSELab();    // Own method
```

The deepest child gets access to ALL ancestors' public/protected members. Inheritance flows all the way up the chain.

Important: This chain can be as long as you want. But be careful — very deep chains become hard to understand and maintain. In practice, 2-3 levels is usually enough.

Type 3: Hierarchical Inheritance

One parent, multiple children. Each child inherits independently from the same parent.

```
    A
   / \
  B   C
```

```
class Student {
   public void markAttendance() {
      System.out.println("Attendance marked");
   }
}

class EngineeringStudent extends Student {
   public void attendLab() {
      System.out.println("Engineering lab attended");
   }
}

class MedicalStudent extends Student {
   public void attendLab() {
      System.out.println("Medical lab attended");
   }
}
```

Both `EngineeringStudent` and `MedicalStudent` inherit from `Student`. They share `markAttendance()` but have their own `attendLab()` implementations.

```
EngineeringStudent es = new EngineeringStudent();
es.markAttendance(); // "Attendance marked"
```

```
es.attendLab();      // "Engineering lab attended"

MedicalStudent ms = new MedicalStudent();
ms.markAttendance(); // "Attendance marked"
ms.attendLab();      // "Medical lab attended"
```

Type 4: Multiple Inheritance — NOT Supported in Java

Multiple inheritance means a class has **more than one parent**.

```
  A    B
   \  /
    C

// This is ILLEGAL in Java:
class C extends A, B { }  // COMPILE ERROR
```

Why does Java not support this?

Because of the **Diamond Problem.**

---

8. THE DIAMOND PROBLEM — Deep Dive

The diamond problem is one of the most famous problems in OOP. Understanding it will deepen your understanding of how the language works.

Setup:

```
      A
      / \
     B   C
      \ /
       D

class A {
   void show() {
      System.out.println("A's show");
   }
}
```

```
class B extends A {
    void show() {
        System.out.println("B's show");  // Overrides A's show
    }
}

class C extends A {
    void show() {
        System.out.println("C's show");  // Also overrides A's show
    }
}

// HYPOTHETICAL (not actually valid Java):
class D extends B, C {
    // D inherits show() from... B? or C?
}
```

Now, when you call:

```
D d = new D();
d.show();  // Which show() runs?? B's or C's?
```

The compiler has NO way to determine which `show()` to call. Both B and C have their own `show()` method. D would have two conflicting `show()` methods — ambiguity at compile time.

This ambiguity is the Diamond Problem. Java's designers chose to simply not support multiple inheritance for classes, eliminating the problem entirely.

The shape is a diamond when you draw it:

```
  A        ← Top of diamond
  / \
 B   C     ← Sides of diamond
  \ /
   D       ← Bottom of diamond
```

How does Java solve this when you need multiple inheritance behavior?

Through **interfaces**. Interfaces have a rule that resolves the ambiguity (we'll cover that in the interfaces chapter). But for classes, multiple inheritance is simply disallowed.

This is why Java is considered "cleaner" than C++ in this regard — C++ allows multiple inheritance but requires complex rules to handle the diamond problem. Java just removes the problem.

---

9. PROPERTIES OF INHERITANCE

Property 1: Code Reusability

Write once, use everywhere. The `start()` method in `Vehicle` is written once, but available in every vehicle subclass.

Property 2: "Is-A" Relationship

Every subclass object IS-A parent class object. This means you can store a subclass object in a parent class reference:

```
Vehicle v = new Car();   // Valid! A Car IS a Vehicle
v.start();               // Calls Car's start (or Vehicle's if Car doesn't override)
```

This is the foundation of polymorphism (next chapter).

Property 3: Method Overriding

A child class can provide its own implementation of a parent's method:

```
class Vehicle {
    void start() {
        System.out.println("Generic vehicle starting");
    }
}

class ElectricCar extends Vehicle {
    void start() {  // Overrides Vehicle's start
        System.out.println("Electric car starting silently");
    }
}

ElectricCar ec = new ElectricCar();
ec.start();  // "Electric car starting silently" — uses OWN version
```

The child's method "shadows" the parent's. If the child doesn't override, the parent's version is used.

Property 4: Supports Polymorphism

Because of inheritance, you can write code that works with the parent type but actually operates on any subclass:

```
void makeVehicleStart(Vehicle v) {
    v.start();  // Could be Car, Truck, Motorcycle — works for all!
}
```

---

10. THE `super` KEYWORD — Complete Deep Dive

`super` is to inheritance what `this` is to the current object. Just as `this` holds a reference to the current object, `super` holds a reference to the parent object.

Use 1: Access Parent Class Variables

When a child class has a variable with the same name as a parent class variable, `super` distinguishes them:

```
class A {
    int x = 10;
}

class B extends A {
    int x = 20;  // Same name as parent's x

    void display() {
        System.out.println(x);       // 20 — local/child's x
        System.out.println(super.x); // 10 — parent's x
    }
}
```

Without `super.x`, you'd always get the child's `x`. With `super.x`, you explicitly access the parent's version.

Use 2: Call Parent Class Methods

When a child overrides a method but still wants to call the parent's version:

```
class Vehicle {
    void start() {
        System.out.println("Checking fuel...");
        System.out.println("Vehicle starting...");
    }
}

class Car extends Vehicle {
    void start() {
        super.start();  // First, do everything Vehicle does
        System.out.println("Car-specific startup: adjusting mirrors...");
    }
}

Car c = new Car();
c.start();
// Output:
// Checking fuel...
// Vehicle starting...
// Car-specific startup: adjusting mirrors...
```

This is extremely useful — you can reuse the parent's logic and then add extra behavior on top.

Use 3: Call Parent Class Constructor (Most Important)

This is the most commonly used form of `super`. It calls the parent's constructor from inside the child's constructor.

```
class Student {
    String name;
    int age;
    int rollNumber;

    // Parameterized constructor
    public Student(String name, int age, int rollNumber) {
        this.name = name;
        this.age = age;
        this.rollNumber = rollNumber;
    }
}

class EngineeringStudent extends Student {
```

```
    String college;

    // Child's constructor
    public EngineeringStudent(String name, int age, int rollNumber, String college) {
        super(name, age, rollNumber);  // Call parent's constructor first!
        this.college = college;         // Then set own field
    }
}

EngineeringStudent es = new EngineeringStudent("Aditya", 20, 101, "IIT Bombay");
System.out.println(es.name);    // Aditya — set by super()
System.out.println(es.age);     // 20 — set by super()
System.out.println(es.college); // IIT Bombay — set by child
```

How it flows:

1. `EngineeringStudent` constructor is called
2. `super(name, age, rollNumber)` jumps into `Student`'s constructor
3. `Student` sets `name`, `age`, `rollNumber`
4. Control returns to `EngineeringStudent`'s constructor
5. `this.college = college` is executed

Critical Rule: `super()` must be the **first statement** in the constructor. You cannot write code before `super()`.

```
public EngineeringStudent(String name, int age, int rollNumber, String college) {
    System.out.println("Hello"); // ERROR! super() must be FIRST
    super(name, age, rollNumber);
    this.college = college;
}
```

What if you don't call `super()`?

Java automatically inserts a call to the parent's **no-argument constructor** (default constructor) if you don't explicitly call `super()`.

```
class Student {
    String name;

    // No explicit constructor — Java creates a default no-arg constructor
}
```

```
class EngineeringStudent extends Student {
    String college;

    public EngineeringStudent(String college) {
        // super() is automatically called here — calls Student's default constructor
        this.college = college;
    }
}
```

The danger: If the parent has only a parameterized constructor (no default constructor), and you don't call `super()` with arguments, you get a compile error:

```
class Student {
    String name;

    public Student(String name) {  // Only parameterized constructor
        this.name = name;
    }
    // Java does NOT create a default constructor since you defined one
}

class EngineeringStudent extends Student {
    public EngineeringStudent() {
        // Java tries to call super() — but Student has no no-arg constructor!
        // COMPILE ERROR!
    }
}
```

Fix: Either call the right `super()`, or add a no-arg constructor to the parent.

---

11. PUTTING IT ALL TOGETHER — A Complete Real Example

Here's a complete example combining encapsulation, access modifiers, packages, inheritance, and `super`:

```
// File: vehicle/Vehicle.java
package vehicle;
```

```java
public class Vehicle {
    private String brand;
    private int year;
    protected int speed;  // Protected — subclasses need this

    public Vehicle(String brand, int year) {
        this.brand = brand;
        this.year = year;
        this.speed = 0;
    }

    public void start() {
        System.out.println(brand + " is starting...");
    }

    public void accelerate(int increase) {
        speed += increase;
        System.out.println("Speed: " + speed);
    }

    public void brake() {
        speed = Math.max(0, speed - 10);
        System.out.println("Braking. Speed: " + speed);
    }

    // Getters
    public String getBrand() { return brand; }
    public int getYear() { return year; }
    public int getSpeed() { return speed; }
}

// File: vehicle/Car.java
package vehicle;

public class Car extends Vehicle {
    private int gear;
    private int numberOfDoors;

    public Car(String brand, int year, int numberOfDoors) {
        super(brand, year);  // Call Vehicle's constructor
        this.gear = 1;
        this.numberOfDoors = numberOfDoors;
    }
```

```java
    public void setGear(int gear) {
        if (gear < 1 || gear > 6) {
            System.out.println("Invalid gear");
            return;
        }
        this.gear = gear;
        System.out.println("Gear set to: " + gear);
    }

    @Override
    public void start() {
        super.start();  // Do everything Vehicle does first
        System.out.println("Car-specific: buckling seatbelt reminder");
    }

    // Getter
    public int getGear() { return gear; }
    public int getNumberOfDoors() { return numberOfDoors; }
}

// File: vehicle/ElectricCar.java
package vehicle;

public class ElectricCar extends Car {
    private int batteryPercent;

    public ElectricCar(String brand, int year, int numberOfDoors, int batteryPercent) {
        super(brand, year, numberOfDoors);  // Call Car's constructor
        this.batteryPercent = batteryPercent;
    }

    @Override
    public void start() {
        if (batteryPercent < 5) {
            System.out.println("Battery too low! Charge first.");
            return;
        }
        super.start();  // Call Car's start (which calls Vehicle's start)
        System.out.println("Electric motor engaged. Battery: " + batteryPercent + "%");
    }

    public void charge(int percent) {
        batteryPercent = Math.min(100, batteryPercent + percent);
        System.out.println("Charging... Battery: " + batteryPercent + "%");
```

```
    }

    public int getBatteryPercent() { return batteryPercent; }
}

// File: Main.java
import vehicle.*;

public class Main {
    public static void main(String[] args) {
        ElectricCar ec = new ElectricCar("Tesla", 2024, 4, 80);

        ec.start();         // Calls ElectricCar's start → Car's start → Vehicle's start
        ec.accelerate(30);  // Inherited from Vehicle
        ec.setGear(3);      // Inherited from Car
        ec.charge(10);      // Own method

        System.out.println("Brand: " + ec.getBrand());          // From Vehicle
        System.out.println("Doors: " + ec.getNumberOfDoors());  // From Car
        System.out.println("Battery: " + ec.getBatteryPercent()); // Own

        // ec.brand = "BMW";  // ERROR — private in Vehicle
        // ec.speed = 200;    // Hmm, speed is protected... this actually works since same package
    }
}
```

Output:

```
Tesla is starting...
Car-specific: buckling seatbelt reminder
Electric motor engaged. Battery: 80%
Speed: 30
Gear set to: 3
Charging... Battery: 90%
Brand: Tesla
Doors: 4
Battery: 90
```

---

12. KEY TAKEAWAYS — Everything in One Place

Encapsulation:

- Bundle data + behavior in one class ✅ (we always do this)
- Make fields `private`, expose through `public` methods ✅ (we need to consciously do this)
- Enables validation, logging, business rules at the access point
- Getters = read access, Setters = write access with validation

Access Modifiers (strictest to loosest):

- `private` → Same class only
- `default` (nothing) → Same package
- `protected` → Same package + subclasses
- `public` → Everyone

Packages:

- Folder that groups related classes
- Declare with `package packagename;` at top of file
- Import with `import packagename.ClassName;`
- Use wildcard `import packagename.*;` for all classes
- Java's built-in packages: `java.lang` (auto), `java.util`, `java.io`, etc.

Inheritance:

- Models "is-a" relationship
- Use `extends` keyword
- Child gets parent's public + protected members
- Does NOT get private members
- Does NOT inherit constructors (but can call with `super()`)

Types:

- Single: A → B ✅ (Java supports)
- Multilevel: A → B → C ✅ (Java supports)
- Hierarchical: A → B and A → C ✅ (Java supports)
- Multiple: A+B → C ❌ (Java doesn't support — Diamond Problem)

`super` keyword:

- `super.variable` → Access parent's variable
- `super.method()` → Call parent's method
- `super()` → Call parent's constructor (must be first line in constructor)

This is the complete, in-depth explanation of every concept covered in the video. Each concept builds on the previous one — encapsulation needs access modifiers, inheritance needs `super`, and everything together forms the foundation for polymorphism and abstraction that come next.